Implementation and Evaluation of Quantum Algorithms for the Clique Problem

Alen Leban, Uroš Čibej

University of Ljubljana; Faculty of Computer and Information Science
Večna pot 113, 1000 Ljubljana, Slovenija
E-mail: al2719@student.uni-lj.si, uros.cibej@fri.uni-lj.si

Abstract

The Maximum Clique Problem is a well-known NP-hard combinatorial optimization problem with numerous practical applications. With advances in quantum computing, several quantum algorithmic paradigms offer promising avenues for solving such problems. In this paper, we present the implementation and evaluation of three distinct quantum approaches: the Quantum Approximate Optimization Algorithm (QAOA), Quantum Annealing (QA), and Grover's search algorithm. We analyze their structural advantages and disadvantages, focusing on solution accuracy, success probability, and resource consumption in terms of qubits and circuit depth.

1 Introduction

Clique problems are key in graph theory, aiming to find the largest or a complete subgraph of specific-size. These problems are widely applicable in social network analysis, bioinformatics, finance, and circuit design. Garey and Johnson [1] show that the decision version is NP-complete and the optimization version is NP-hard, making exact solutions for large instances difficult on classical computers, leading researchers to explore alternative paradigms.

Quantum computing offers several approaches to solving such combinatorial problems, including heuristic gate-based methods (QAOA), analog optimization (QA), and exact search algorithms such as Grover's search. Although these three quantum paradigms are well-understood theoretically, their practical performance, resource needs, and trade-offs vary when applied to specific graph problems. Literature often analyzes these methods in isolation or on broad benchmarks. Cross-paradigm evaluations are vital to understand how graph features such as density, size, and constraints impact circuit depth, qubit usage, and success probability.

This paper presents a comparative implementation and evaluation of QAOA, QA, and Grover's algorithm for the clique problem. We map problems to quantum formulations, using Quadratic Unconstrained Binary Optimization (QUBO) for QAOA and QA, and a custom oracle for Grover's. Evaluation focuses on solution accuracy, success probability, and resource use measured by qubits and circuit depth.

The remainder of this paper is organized as follows. Section 2 establishes the mathematical preliminaries of the clique problem and provides an overview of the three quantum algorithmic paradigms. Section 3 details our specific mapping strategies and adaptations for the clique problem across each algorithm. Section 4 describes our empirical experimental setup and discusses the evaluation results. Finally, Section 5 concludes the paper and outlines avenues for future work.

2 Preliminaries

Given an undirected graph $G = (V, E)$, where V is the set of vertices (nodes) and E is the set of edges (connections), a clique is a subset of vertices $C \subseteq V$ such that every pair of distinct vertices in C is adjacent (connected by an edge). In other words, the subgraph induced by C is a complete graph.

In this paper, we will deal mostly with the decision and optimization versions of the problem - hereafter referred to as `KClique` and `MaxClique`, respectively.

The clique problems are computationally hard [1], under current knowledge, thus untractable even with quantum computers. However, a few quantum techniques show promising potential for practical speedups; three of these will be examined here.

2.1 Quantum Approximate Optimization Algorithm (QAOA)

A leading approach to solving combinatorial problems on gate-based quantum computers is Quantum Approximate Optimization Algorithm, introduced by Fahri et al. [2] that approximates adiabatic evolution by alternating applications of a mixer Hamiltonian H_M and a problem Hamiltonian H_P. A depth-p quantum circuit contains $2p$ variational parameters (β_i, γ_i) optimized by a classical optimizer. The mixer Hamiltonian promotes exploration of the search space by driving transitions between candidate solutions, while the problem Hamiltonian is constructed such that its ground state corresponds to the optimal solution of the combinatorial problem. A common approach for encoding combinatorial optimization problems into a problem Hamiltonian is the QUBO formulation, in which the objective function is expressed using binary decision variables $x_i \in \{0, 1\}$. To implement the

problem on a quantum computer, these binary variables are mapped to qubit operators using the transformation

$$x_i = \frac{1 - Z_i}{2}, \tag{1}$$

where Z_i denotes the Pauli-Z operator acting on qubit i. Since the eigenvalues of Z_i are +1 and -1, this mapping allows the QUBO objective function to be rewritten as a Hamiltonian composed of Pauli-Z terms whose ground state encodes the optimal solution.

2.2 Quantum Annealing

Quantum Annealing is a quantum optimization technique closely related to Adiabatic Quantum Computation (AQC). The system evolves from an initial Hamiltonian to a problem Hamiltonian whose ground state encodes the solution. According to the adiabatic theorem, sufficiently slow evolution keeps the system in the ground state throughout the process. The contributions of the two Hamiltonians are controlled by the annealing schedule through weighting functions $A(s)$ and $B(s)$, where $s \in [0, 1]$ is the normalized annealing time.

$$H = -\frac{A(s)}{2}\left(\sum_i \hat{\sigma}_x^{(i)}\right) + \frac{B(s)}{2}\left(\sum_i h_i \hat{\sigma}_z^{(i)} + \sum_{i>j} J_{ij}\hat{\sigma}_z^{(i)}\hat{\sigma}_z^{(j)}\right) \tag{2}$$

2.3 Grover

As QAOA and QA are heuristic algorithms, they don't have probabilistic guarantees for finding optimal solutions. As a contrast, we wanted to include Grover's algorithm [3] as the exact quantum method.

Grover's algorithm can be used to search a space of $N = 2^n$ elements in $O(\sqrt{N})$ oracle evaluations. Each iteration applies an oracle that phase-marks valid states followed by the Grover diffusion operator, amplifying the probability of measuring marked solutions.

3 Adaptation for cliques

3.1 Clique search problem QUBO mapping

We implemented QUBO formulations for the `MaxClique` optimization problem and the `KClique` decision problem. Both formulations are based on the observation that a clique in a graph G corresponds to an independent set in its complement graph $\overline{G}$. To enforce clique constraints in G, we add penalty terms for each edge in $\overline{G}$ as independence constraints for the independent set problem. For `MaxClique`, the objective is to maximize the number of selected vertices while penalizing violations of the independence constraint (Equation (3)). For `KClique`, additional constraints enforce the selection of exactly K vertices and the required number of edges among them (Equation (4)).

$$H_{max} = -A\sum_{v\in V(\overline{G})} x_v + B\sum_{(u,v)\in E(\overline{G})} x_u x_v \tag{3}$$

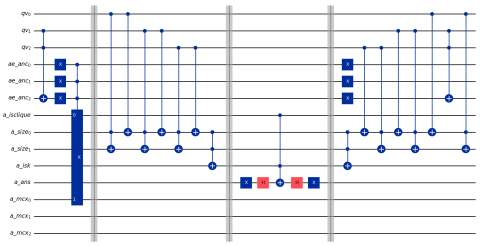


Figure 1: Oracle circuit for checking if a subset of vertices is a k-clique on a line graph with 3 vertices. The four parts separated by barriers do the following: check for clique violations (on missing edges), check if a solution has k vertices, mark the elements by flipping the phase, and uncompute all operations on the ancilla qubits.

$$H_k = A\left(K - \sum_{v\in V(\overline{G})} x_v\right)^2 + B\left[\frac{K(K-1)}{2} - \sum_{(u,v)\in E(\overline{G})} x_u x_v\right] \tag{4}$$

3.2 QA for cliques

Similarly to the QUBO encoding used for QAOA, the objective function of QA is also formed around the independent set problem. For the `MaxClique` problem, the linear terms encourage selecting more vertices, while the quadratic terms penalize breaking independence constraints on edges of the complementary graph (Equation (5)). For the `KClique` problem, the first part of the function is replaced with a penalty for deviating from the target clique size k (Equation (6)).

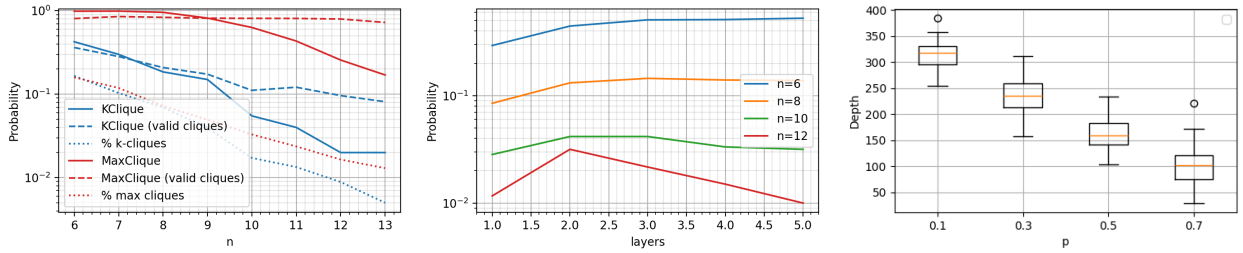
$$H_{max} = -\sum_{i=1}^{|V|} x_i + 2\sum_{(i,j)\in\overline{E}} x_i x_j \tag{5}$$

$$H_k = \left(k - \sum_i x_i\right)^2 + \sum_{(i,j)\in\overline{E}} x_i x_j \tag{6}$$

3.3 Clique search with Grover

Grover's algorithm can be used to search arbitrary binary strings among all n-bit strings as long as we are able to implement an efficient oracle that can mark those searched strings. We chose to implement an oracle for solving the `KClique` problem only as for the `MaxClique` you can simply run the circuit multiple times with increasing k.

The logic for checking k-cliques in our oracle was made from two parts similarly to [4]: the first half checks for clique violations by ensuring no two unconnected vertices are chosen together, while the second part checks if the clique size is exactly k. A graph's structure is embedded in the choices of gate positions, e.g. checking for violations only among disconnected vertices. The `a_isclique` ancilla qubit is flipped to 1 when there are no clique violations between pairs of vertices, while the



(a) (b) (c)

Figure 2: **(a)** The probability of finding cliques at different numbers of nodes in the graph n. Dashed lines represent the probability of a run producing a valid clique, while dotted lines represent the percentage of valid cliques among all vertex subsets of size k. **(b)** Finding the optimal number of QAOA layers for different graph sizes (n) for the `KClique` problem. At $n = 6$ the peak appears to lie beyond 5 layers, for $n = 8$ and 10 around 3 layers, and for $n = 12$ around 2 layers. **(c)** Transpiled circuit depth for the `MaxClique` problem as a function of graph density p.

`a_isk` ancilla qubit is flipped to 1 when the size of the clique is exactly k. When those two qubits are both 1, we apply an MCX gate to the `q_anc` qubit in the $|-\rangle$ state, marking k-cliques by flipping the phase. A visualization of a circuit is drawn in Figure 1.

4 Experiment setup and results

For testing, we used randomly generated graphs according to the $G(n, p)$ Erdös-Renyi (ER) model. For the `KClique` problem instances, we first generated $G(n, p)$ graphs, and then manually hid a k-clique by uniformly randomly choosing k vertices and fully connecting them to form a k-clique to guarantee a solution exists. Unless specified, the injected clique size for the `KClique` problem was determined as two less than the expected largest clique of an ER model according to a formula from [5]. For the `MaxClique` problem, we used a classical algorithm to determine the size of the largest clique in the generated graph.

The main metric was the probability that the algorithm finds the correct solution (i.e., a valid clique of the correct size), and we sometimes also plot the probability that the solution represents only a valid clique in cases where the solution has an incorrect number of vertices.

4.1 QAOA

Due to the exponential difficulty of simulating quantum circuits on a classical computer, our tests used graphs with up to 15 nodes.

Experiments test different graph sizes, densities, target clique sizes k, numbers of QAOA layers, runs per graph, circuit depths produced, and the required number of optimiser function evaluations.

As expected, the difficulty of both `KClique` and `MaxClique` scales exponentially in the size of the problem at a fixed number of iterations as seen in Figure 2a although the difficulty of `MaxClique` seems to start dropping slowly at first, possibly due to having edges and triangles as their largest cliques which are very common subgraph structures. The success probability seems to scale proportionally with the percentage of possible correct solutions. The success probability of the `MaxClique` problem is higher, possibly due to the high probability of returning a valid clique at all tested problem sizes.

While testing different amounts of QAOA layers on different graph sizes, we noticed that smaller graphs benefited more from having more layers than larger graphs (Figure 2b) although the reason for that remains unclear.

Very dense or very sparse graph instances showed higher solution probabilities than those with a density around 0.4, possibly because they had more correct solutions. With a noisy backend, however, sparse graphs also posed a big problem because of a larger amount of Pauli operators in the problem Hamiltonian, which increased the depth of the circuit (Figure 2c).

4.2 Quantum Annealing

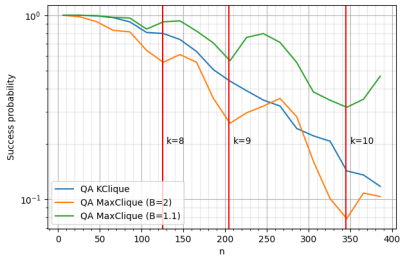
For quantum annealing, we could test on much larger graphs of up to 1000 nodes due to it being a much simpler process to simulate. The majority of experiments were done using the D-Wave's `SimulatedAnnealingSampler` class and later compared with a noisy version, the `PathIntegralAnnealingSampler` class.

QA also shows and exponential decrease in the probability of finding the correct solutions for both clique problems, where `MaxClique` is again being solved more successfully than `KClique` (Figure 3a). While previous work recommends $B = 2$ [6], we found $B = 1.1$ (`MaxClique`) and $B = 1$ (`KClique`) to perform best on our benchmark graphs. A similar parameter search for B was done for the `KClique` problem, where we found $B = 1$ to be optimal. The noisy sampler performed with lower success probability as visible in Figure 3b.

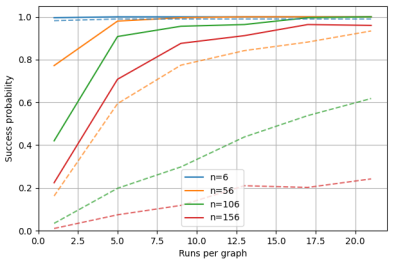
The success probability plot (Figure 3a) for the `MaxClique` problem has unusually distinct peaks and drops at approximately $n = 75, 125, 205, 345$, which corresponds with transitions of average maximum clique sizes to the next whole number. A more detailed analysis of the properties of the generated graphs could help improve our understanding for such behavior.

4.3 Grover's algorithm

Just like QAOA, the circuits produced by Grover's algorithm were also simulated both locally on the `Aer-`



(a)



(b)

Figure 3: **(a)** Success probability of QA when solving both clique problems with four runs on each graph. Success probability for solving the `MaxClique` problem is shown for both the initial and optimized values of the penalty parameter B. Red vertical lines indicate the average size k of the largest clique for each graph size n. **(b)** Success probability for solving the `KClique` problem as the number of runs per graph increases on the `SimulatedAnnealingSampler` (solid line) and `PathIntegralAnnealingSampler` (dashed line).

Table 1: Circuit depth for different graph sizes n and edge counts $|E|$, for 1 iteration of Grover's algorithm.

n	$\|E\|$	depth
2	1	389
3	1	717
3	2	611
3	3	570

`Simulator`, `FakeBrisbane` fake backend and a real backend (`ibm_fez`) provided by IBM Quantum Platform[1].

Already on small instances ($n = 3$) the produced circuits had substantial depth and the amount of noise was close to covering the correct solutions (Figure 4). Denser graphs produce shallower circuits since there are less clique violation checks on disconnected vertices (Table 1). In theory, for the 3 vertex, fully connected graph the ideal number of Grover iterations would be at around 2, but because running deeper circuits would accumulate too much noise, we decided to perform all tests using 1 iteration only.

5 Conclusion and Future Work

We implemented and evaluated QAOA, QA, and Grover's algorithm for solving the `KClique` and `MaxClique` problems. While all three methods successfully solved small instances, their scalability on current quantum hardware is primarily limited by circuit depth (especially for Grover's algorithm) and available qubits.

Future work could improve QAOA by replacing the default initial state and mixer with problem-specific alternatives, such as Dicke-state preparation or a Quantum Alternating Operator Ansatz [7], restricting the search to valid solutions. For QA, analyzing the minimum spectral gap and exploring modified Hamiltonians such as trigger Hamiltonians [8] could provide further insight into instance difficulty. Finally, designing shallower oracle circuits could improve the practical scalability of Grover's algorithm.

[1] https://quantum.cloud.ibm.com/

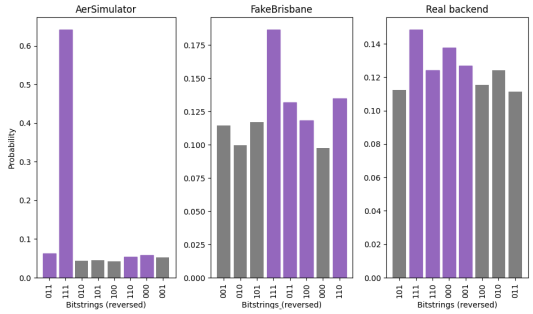


Figure 4: Probabilities of measuring each bitstring at the end of 1 iteration of Grover's algorithm for solving the `KClique` problem on different backends. The graph we used here was fully connected with 3 vertices with exactly 1 correct solution `111` ($k = 3$).

References

[1] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco: W. H. Freeman and Company, 1979. ISBN: 0-7167-1045-5.

[2] E. Farhi et al. *A Quantum Approximate Optimization Algorithm*. 2014. arXiv: 1411.4028 [quant-ph]. URL: https://arxiv.org/abs/1411.4028.

[3] L. K. Grover. *A fast quantum mechanical algorithm for database search*. 1996. arXiv: quant-ph/9605043 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/9605043.

[4] A. Haverly and S. López. "Implementation of Grover's Algorithm to Solve the Maximum Clique Problem". In: *2021 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. 2021, pp. 441–446. DOI: 10.1109/ISVLSI51109.2021.00087.

[5] W. Yang et al. *Lecture 2: Largest Clique in a Random Graph*. 2018.

[6] A. Gherardi and A. Leporati. *An Analysis of Quantum Annealing Algorithms for Solving the Maximum Clique Problem*. 2024. arXiv: 2406.07587 [quant-ph]. URL: https://arxiv.org/abs/2406.07587.

[7] S. Hadfield et al. "From the Quantum Approximate Optimization Algorithm to a Quantum Alternating Operator Ansatz". In: *Algorithms* 12.2 (Feb. 2019), p. 34. ISSN: 1999-4893. DOI: 10.3390/a12020034. URL: http://dx.doi.org/10.3390/a12020034.

[8] V. Mehta et al. "Quantum annealing with trigger Hamiltonians: Application to 2-satisfiability and nonstoquastic problems". In: *Physical Review A* 104.3 (2021). ISSN: 2469-9934. DOI: 10.1103/physreva.104.032421. URL: http://dx.doi.org/10.1103/PhysRevA.104.032421.